

目录

一 实验目的及内容	4
二 实验原理及技术路线	4
2.1 概述	4
2.2 实验原理	4
2.2.1 图像分类原理	4
2.2.2 迁移学习与预训练模型	5
2.2.3 语义分割与前景 mask	5
2.2.4 损失函数与评价指标	6
2.3 技术路线	6
三 所用仪器、材料	8
3.1 实验环境	8
3.2 数据集	8
3.3 主要软件库	9
3.4 模型资源与输出材料	9
四 实验方法、步骤	10
4.1 实验方法	10
4.1.1 数据划分方法	10
4.1.2 图像预处理与 mask 转换方法	11
4.1.3 模型训练方法	12
4.1.4 评价与可视化方法	13
4.2 实验步骤	14
4.3 实验代码	15
4.3.1 数据处理模块重要代码	15
4.3.2 分类模型模块重要代码	16
4.3.3 分割模型模块重要代码	17
4.3.4 训练与评估模块重要代码	18
五 实验过程原始记录	19
5.1 数据样例与类别分布记录	19
5.2 Mask 转换与预处理检查记录	21
5.3 训练过程记录	23
5.4 测试数据与计算结果记录	24
5.5 模型输出样例记录	26

5.6	实验记录索引	28
六	实验结果、分析和结论	28
6.1	分类结果	29
6.2	分割结果	31
6.3	轻量消融实验	33
6.4	失败案例分析	34
6.5	结论与改进方向	34

一、实验目的及内容

上机 3 计算机视觉综合实践

上机目的：通过上机，掌握计算机视觉综合实践应用。

上机内容：任选本课程所讲的计算机视觉，不限数据集和技术内容，尽量使用多种技术进行实践。建议包含数据预处理、模型微调、评估与可视化。

要求：完成自选内容的模型的训练和评估，运行出正确的结果。上传源代码和上机报告电子版，并双面打印上机报告纸质版，作为期末课程报告。

二、实验原理及技术路线

2.1 概述

本实验以 Oxford-IIIT Pet 数据集为实验对象，围绕宠物品种分类和宠物前景语义分割两个任务展开。分类任务要求模型根据输入图像判断宠物所属的品种类别，共包含 37 类猫狗品种；分割任务要求模型在像素级别区分宠物前景和图像背景，从而得到宠物主体区域的二分类分割结果。通过这两个任务，可以从图像级识别和像素级理解两个角度，进一步掌握卷积神经网络在计算机视觉任务中的基本应用流程。

本实验整体采用迁移学习思想。由于 Oxford-IIIT Pet 数据集规模相对有限，如果完全从零开始训练深层神经网络，模型可能会出现收敛较慢、泛化能力不足等问题。因此，本实验优先使用在大规模图像数据集上预训练的模型作为基础，再根据宠物分类和前景分割任务修改输出层并进行微调。分类任务中，使用 ResNet18 作为 baseline，使用 EfficientNet-B0 作为主要分类模型；分割任务中，使用 DeepLabV3-MobileNetV3 作为主要语义分割模型。

本实验流程主要包括数据读取、固定划分、图像预处理、模型构建、模型训练、测试集评估和结果可视化。分类任务使用 Accuracy、Macro-F1、Weighted-F1 和 Top-5 Accuracy 等指标评价模型识别性能；分割任务使用 Pixel Accuracy、IoU、mIoU 和 Dice 等指标评价像素级预测效果。最后结合训练曲线、混淆矩阵、分类样例和分割可视化结果，对实验结果进行整理和分析。

2.2 实验原理

2.2.1 图像分类原理

图像分类是计算机视觉中的基础任务，其目标是根据输入图像预测其所属类别。对于本实验中的宠物品种分类任务，输入为一张宠物图像，输出为 37 个宠物品种类别中的一个类别。分类模型通常由特征提取部分和分类输出部分组成。特征提取部分通过卷积、非线性激活、池化等操作提取图像中的边缘、纹理、颜色、形状以及高层语义特征；分类输出部分则将提取到的特征映射为各类别的预测得分。

在神经网络分类模型中，最后一层通常输出一组 logits，其维度等于类别数量。本实验的分类任务共有 37 个类别，因此 ResNet18 和 EfficientNet-B0 的最终分类层均被

修改为 37 维输出。模型输出经过 softmax 后可以得到各类别的预测概率，概率最大的类别即为模型的预测结果。对于宠物品种分类而言，不同类别之间可能具有相似的毛色、体型和姿态，因此模型不仅要学习整体轮廓，还需要关注面部结构、耳朵形状、毛发纹理等较细致的视觉特征。

2.2.2 迁移学习与预训练模型

迁移学习是指将一个任务中学习到的知识迁移到另一个相关任务中使用。在深度学习图像任务中，常见做法是使用在 ImageNet 等大规模数据集上预训练的模型作为初始化，然后在目标数据集上继续训练。预训练模型已经学习到较通用的低层视觉特征和部分高层语义特征，因此在目标数据集规模较小时，迁移学习可以减少训练难度，加快模型收敛，并在一定程度上提升模型的泛化能力。

本实验分类部分使用 ResNet18 和 EfficientNet-B0。ResNet18 通过残差连接缓解深层网络训练过程中的梯度退化问题，使网络能够更稳定地学习图像特征。由于 ResNet18 结构清晰、训练速度较快，本实验将其作为分类 baseline。EfficientNet-B0 通过复合缩放思想同时平衡网络深度、宽度和输入分辨率，在参数规模较小的情况下仍具有较好的特征表达能力，因此作为本实验的主要分类模型。

在模型修改方面，两个分类模型均加载 ImageNet 预训练权重，并将原始分类层替换为适用于 37 类宠物品种分类的新输出层。训练过程中，模型在保留预训练视觉特征的基础上，根据 Oxford-IIIT Pet 数据集进一步调整参数，使其更适合宠物品种识别任务。

2.2.3 语义分割与前景 mask

语义分割是像素级视觉理解任务，其目标是为图像中的每个像素预测类别标签。与图像分类只输出整张图像的类别不同，语义分割需要输出与输入图像空间位置对应的预测结果，因此更关注物体边界、局部区域和空间结构信息。在本实验中，分割任务的重点不是判断宠物品种，而是从图像中提取宠物主体区域。

本实验的分割任务为宠物前景二分类语义分割，即将图像中的每个像素划分为背景或宠物前景。Oxford-IIIT Pet 数据集提供 trimap 标注，原始标注中包含宠物主体、边界区域和背景区域。为了使任务更加聚焦，本实验将原始 trimap 转换为二值 mask：背景像素映射为 0，宠物主体和边界区域统一映射为前景 1。经过该转换后，分割模型只需要输出背景和宠物前景两个类别。

分割模型使用 DeepLabV3-MobileNetV3。DeepLabV3 通过空洞卷积扩大感受野，并结合多尺度上下文信息提升语义分割能力；MobileNetV3 作为轻量化 backbone，可以在保证一定特征表达能力的同时降低计算量。对于本实验的宠物前景提取任务，该模型既能完成像素级预测，又能控制训练和推理的计算成本，适合课程实验环境使用。

在分割任务的数据处理过程中，需要保证图像和 mask 的空间变换保持一致。例如，对图像和 mask 进行缩放、裁剪或翻转时，二者必须同步变化，否则会导致像素

标签与图像内容错位。同时，mask 本身是离散类别标签，因此在调整 mask 尺寸时应使用最近邻插值，避免产生不属于任何类别的非法像素值。

2.2.4 损失函数与评价指标

本实验的分类任务和分割任务均使用交叉熵损失进行训练。分类任务中，交叉熵损失用于度量模型预测类别分布与真实类别标签之间的差异。设一个 batch 中共有 N 个样本，第 i 个样本真实类别为 y_i ，模型对该真实类别预测的概率为 p_{i,y_i} ，则分类交叉熵损失可表示为：

$$\mathcal{L}_{cls} = -\frac{1}{N} \sum_{i=1}^N \log p_{i,y_i}$$

分割任务中，交叉熵损失在像素级别计算。模型需要对每个像素预测其属于背景或宠物前景的概率，损失函数则衡量每个像素预测结果与真实 mask 标签之间的差异。通过最小化像素级交叉熵，模型逐渐学习宠物主体区域与背景区域之间的边界和语义差异。

分类任务的评价指标包括 Accuracy、Macro-F1、Weighted-F1 和 Top-5 Accuracy。Accuracy 表示整体分类正确率，能够反映模型在全部测试样本上的总体表现；Macro-F1 对每个类别的 F1 值进行平均，可以反映不同类别上的平均识别能力，适合观察类别间表现是否均衡；Weighted-F1 按类别样本数加权，更接近整体样本分布下的综合表现；Top-5 Accuracy 则用于衡量真实类别是否出现在模型预测概率最高的前 5 个类别中。

分割任务的评价指标包括 Pixel Accuracy、IoU、mIoU 和 Dice。Pixel Accuracy 表示所有像素中预测正确的比例；IoU 表示预测区域与真实区域的交集和并集之比，可用于衡量某一类别区域的重合程度；mIoU 是各类别 IoU 的平均值，是语义分割任务中常用的综合指标；Dice 系数同样用于衡量预测前景与真实前景的重合程度，对前景区域分割效果具有较直观的解释意义。

2.3 技术路线

本实验的技术路线从数据集准备开始，依次完成数据处理、模型训练、模型评估和结果分析。整体来看，分类任务和分割任务共享 Oxford-IIIT Pet 数据集，但二者使用的标签形式和输出目标不同：分类任务使用宠物品种标签，输出图像级类别；分割任务使用 trimap 标注转换得到的二值 mask，输出像素级前景预测结果。

实验首先通过 TorchVision 提供的 Oxford-IIIT Pet 数据集接口读取图像、分类标签和分割标注。为了保证实验结果可复现，使用固定随机种子对官方 trainval 部分进行训练集和验证集划分，并保留官方 test 部分作为最终测试集。随后，分类任务和分割任务分别构建对应的数据预处理流程。分类任务主要进行图像缩放、归一化和训练阶段的数据增强；分割任务则需要在图像预处理之外完成 trimap 到二值 mask 的转换，并保证图像与 mask 的空间对应关系。

分类模型训练分为 **baseline** 和主模型两条路线。首先训练 ResNet18 **baseline**，用于建立基础对比结果；随后训练 EfficientNet-B0 主分类模型，并与 ResNet18 在相同数据划分和测试集条件下进行比较。两个模型均加载 ImageNet 预训练权重，并将分类输出层替换为 37 类。训练完成后，在测试集上计算分类指标，并生成混淆矩阵、正确分类样例和错误分类样例，用于后续结果分析。

分割任务使用 DeepLabV3-MobileNetV3 完成宠物前景预测。模型输出背景和宠物前景两个类别，训练过程中使用二值前景 **mask** 作为监督信号。模型训练完成后，在测试集上计算 Pixel Accuracy、IoU、mIoU 和 Dice 等指标，并生成原始图像、真实 **mask** 和预测 **mask** 的对比图，用于观察模型对宠物主体、边界区域和复杂背景的处理效果。

整体技术路线如图 1 所示。该流程图将实验分为分类任务和分割任务两条路线：分类路线主要完成图像预处理、分类模型训练和分类指标评估；分割路线主要完成 **trimap** 转换、前景分割模型训练和分割指标评估。两条路线最终汇总为实验结果分析和报告撰写材料。

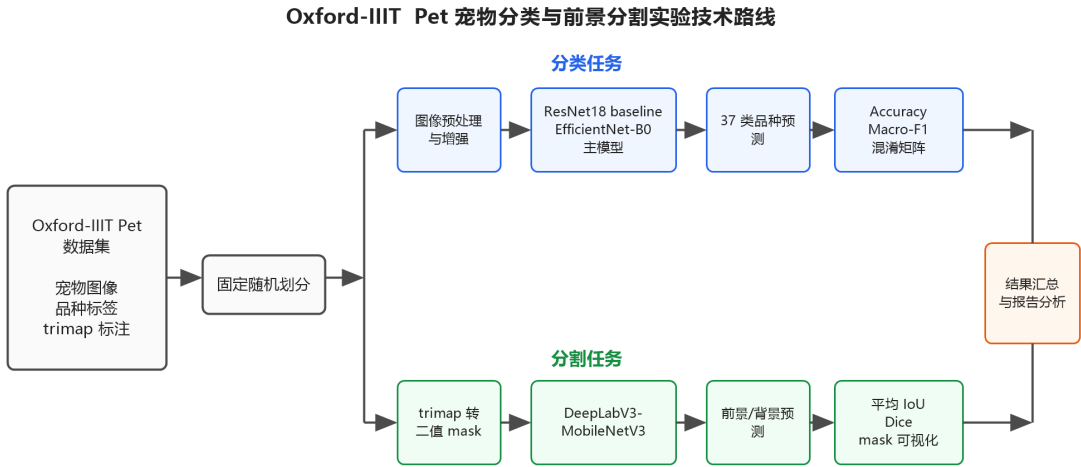


图 1: 实验技术路线图

综上，本实验的技术路线以 Oxford-IIIT Pet 数据集为基础，将宠物品种分类和宠物前景分割作为两个相互关联的视觉任务展开。分类任务侧重图像级语义识别，分割任务侧重像素级区域理解；二者共同构成从数据处理、模型训练、指标评估到可视化分析的完整实验流程。

三、所用仪器、材料

3.1 实验环境

本实验在本地计算机和远程服务器上共同完成。实验代码的调试、结果整理和报告材料生成主要在 Windows 本地环境中完成；模型训练和部分计算量较大的实验主要在远程服务器上完成。训练、评估和可视化程序均使用 Python 编写，深度学习模型基于 PyTorch 框架实现，并通过 CUDA 调用 GPU 进行加速。主要实验环境如表 1 所示。

表 1: 实验环境

项目	配置
本地操作系统	Windows
本地 Python 环境	conda 环境 unet
本地 Python 路径	D:\Anaconda3\envs\unet\python.exe
Python 版本	Python 3.10.20
本地 GPU	NVIDIA GeForce RTX 4060 Laptop GPU
远程服务器 GPU	NVIDIA GeForce RTX 4090 24GB
CUDA	CUDA 12.6
深度学习框架	PyTorch 2.11.0+cu126
视觉工具库	TorchVision 0.26.0+cu126

3.2 数据集

本实验使用 Oxford-IIIT Pet 数据集作为主要实验数据。该数据集包含猫和犬的宠物图像，共有 37 个宠物品种类别，并同时提供图像分类标签和语义分割 trimap 标注。在分类任务中，模型以宠物图像作为输入，输出对应的 37 类宠物品种标签；在分割任务中，本实验将原始 trimap 标注转换为二值 mask，其中背景记为 0，宠物前景记为 1，用于训练宠物前景二分类语义分割模型。

表 2: 实验数据

项目	内容
数据集名称	Oxford-IIIT Pet
数据目录	data/step6_data/oxford-iiit-pet
类别数量	37 类
分类标注	宠物品种类别标签
分割标注	trimap 标注
分割任务设置	背景/宠物前景二分类
随机种子	42

数据划分情况如表 3 所示。官方 `trainval` 部分按照固定随机种子 42 划分为训练集和验证集，官方 `test` 部分只用于最终测试评估，不参与模型训练和模型选择。

表 3: 数据集划分

划分	样本数	用途
train	2944	模型训练
val	736	模型选择与验证
test	3669	最终测试评估

3.3 主要软件库

本实验主要使用 PyTorch 和 TorchVision 完成模型构建、数据读取、迁移学习和训练评估。数据统计、指标计算和结果可视化过程使用 NumPy、pandas、scikit-learn 和 Matplotlib 完成。主要软件库及用途如表 4 所示。

表 4: 主要软件库

软件或库	版本	主要用途
PyTorch	2.11.0+cu126	模型构建、训练、损失计算和 GPU 加速
TorchVision	0.26.0+cu126	数据集读取、图像变换、预训练模型加载
NumPy	1.23.5	数值计算和数组处理
pandas	2.3.3	训练日志、预测结果和指标表格保存
scikit-learn	1.7.2	分类指标、Macro-F1 和混淆矩阵计算
Matplotlib	3.6.2	训练曲线、类别分布、混淆矩阵和分割结果可视化

3.4 模型资源与输出材料

本实验使用的模型均基于 PyTorch/TorchVision 实现。分类任务中，ResNet18 作为基础对比模型，EfficientNet-B0 作为主要分类模型，两者均使用 ImageNet 预训练权重进行迁移学习；分割任务中，使用 DeepLabV3-MobileNetV3 完成宠物前景提取。

表 5: 实验模型资源

任务	模型	用途
图像分类	ResNet18	37 类宠物品种分类基础对比模型
图像分类	EfficientNet-B0	37 类宠物品种分类主模型
语义分割	DeepLabV3-MobileNetV3	宠物前景二分类语义分割主模型

实验过程中生成的训练日志、指标文件、可视化图片和模型权重统一保存在 `outputs/` 目录下，便于实验过程复核和报告材料整理。其中，模型权重属于训练

过程中产生的中间文件，主要用于后续评估和复现实验指标；由于权重文件体积较大，最终项目打包提交时不包含模型权重文件。

表 6: 实验输出材料

输出目录	保存内容
outputs/checkpoints/	分类模型和分割模型的最佳权重，作为训练中间文件使用，最终打包提交时不包含该目录中的权重文件
outputs/logs/	训练过程日志 CSV 文件
outputs/figures/	数据样例、训练曲线、混淆矩阵、分割可视化图片
outputs/results/	测试指标、预测结果、数据划分记录和分析文件

综上，本实验在本地计算机与远程服务器相结合的环境下完成，并在 GPU 加速条件下开展 Oxford-IIIT Pet 数据集上的宠物品种分类和宠物前景语义分割实验。实验使用的数据、日志、指标文件和可视化图片均在项目目录中保存，可用于实验报告撰写和结果复查。

四、实验方法、步骤

本实验围绕 Oxford-IIIT Pet 数据集完成宠物品种分类和宠物前景语义分割两个任务。分类任务要求模型根据输入图像判断宠物所属的 37 个品种类别；分割任务要求模型将图像中的宠物主体从背景中区分出来，得到前景和背景二分类 mask。实验整体流程包括数据读取与固定划分、图像预处理、模型构建、模型训练、测试集评估、可视化结果生成和报告材料整理。

4.1 实验方法

本实验采用“分类模型对比+前景语义分割”的综合实验方案。分类部分使用 ResNet18 作为 baseline,使用 EfficientNet-B0 作为主分类模型;分割部分使用 DeepLabV3-MobileNetV3 作为主分割模型。模型构建、训练和评估均基于 PyTorch 和 TorchVision 完成，并在数据划分和训练过程中设置固定随机种子，以提高实验结果的可复现性。

4.1.1 数据划分方法

本实验通过 TorchVision 提供的 OxfordIIITPet 接口读取数据集。分类任务读取 `target_types="category"`,获得 37 类宠物品种标签;分割任务读取 `target_types="segmentation"` 获得数据集提供的 trimap 标注。

Oxford-IIIT Pet 数据集自带 `trainval` 和 `test` 两个官方划分。本实验没有直接将 `trainval` 全部用于训练，而是使用固定随机种子 42 将其进一步划分为训练集和验证集，其中验证集用于训练过程监控和模型选择。官方 `test split` 保持独立，只在最终评估阶段使用，不参与训练和调参。这样可以保证测试结果具有相对客观的评价意义。

数据划分逻辑由 `src/data_loaders.py` 实现。代码中使用固定种子的 `torch.Generator` 对官方 `trainval` 索引进行随机划分，关键代码如下：

```
1 def make_train_val_indices(length: int, val_ratio: float = 0.2, seed: int = SEED):
2     val_size = int(round(length * val_ratio))
3     train_size = length - val_size
4     generator = torch.Generator().manual_seed(seed)
5     train_subset, val_subset = torch.utils.data.random_split(
6         range(length), [train_size, val_size], generator=generator
7     )
8     return list(train_subset.indices), list(val_subset.indices)
```

Listing 1: 固定训练集和验证集划分代码

划分后的索引保存到 `outputs/results/split_indices_seed42.json`。后续训练、评估和报告复查均可依据该文件使用同一组训练集、验证集和测试集样本。

4.1.2 图像预处理与 mask 转换方法

分类任务和分割任务的输入形式不同，因此本实验分别设计了对应的预处理方式。分类模型输入尺寸设为 224×224 ，训练集使用随机裁剪、随机水平翻转和颜色扰动等轻量增强，验证集和测试集使用 `resize`、`center crop` 和 ImageNet mean/std 归一化。这样既可以增加训练样本的变化，也可以保证验证和测试阶段输入形式稳定。

分割任务输入尺寸设为 320×320 。由于分割 `mask` 表示离散类别标签，图像 `resize` 使用双线性插值，而 `mask resize` 使用最近邻插值，避免插值过程中产生 0、1、2、3 之外的非法类别值。训练阶段的随机裁剪和水平翻转同时作用在图像和 `mask` 上，从而保证像素级标签与图像内容保持一致。

Oxford-IIIT Pet 的原始 `trimap` 标注包含 3 类像素值。本实验将分割任务处理为宠物前景二分类问题，因此需要先将 `trimap` 转换为二值 `mask`。转换规则为：原始值 2 映射为背景 0，原始值 1 和 3 映射为前景 1。该规则由 `src/data_transforms.py` 中的 `convert_trimap_to_binary_mask` 实现：

```
1 def convert_trimap_to_binary_mask(mask):
2     if isinstance(mask, torch.Tensor):
3         invalid = ~(mask == 1) | (mask == 2) | (mask == 3)
4         if bool(invalid.any().item()):
5             values = sorted(int(value) for value in torch.unique(mask).tolist())
6             raise ValueError(f"Unexpected trimap values: {values}")
7         return torch.where(mask == 2, torch.zeros_like(mask), torch.ones_like(mask)).long()
8
9     mask_array = np.asarray(mask)
```

```

10     valid = np.isin(mask_array, [1, 2, 3])
11     if not bool(valid.all()):
12         values = sorted(int(value) for value in np.unique(mask_array).tolist())
13         raise ValueError(f"Unexpected trimap values: {values}")
14     return np.where(mask_array == 2, 0, 1).astype(np.int64)

```

Listing 2: trimap 转二值 mask 代码

该函数在转换前检查原始 mask 像素值是否合法，转换后输出只包含 0 和 1 两类标签。后续分割模型使用交叉熵损失进行二分类像素预测，不再单独设置忽略类别。

4.1.3 模型训练方法

分类任务采用迁移学习方法。ResNet18 和 EfficientNet-B0 均通过 TorchVision 加载 ImageNet 预训练权重，并将最后输出层替换为 37 类输出。ResNet18 结构相对简单、训练速度较快，因此作为 baseline；EfficientNet-B0 在模型规模和特征表达能力之间较为均衡，因此作为本实验的主分类模型。

分类模型的构建逻辑集中在 `src/models_cls.py`。对于 ResNet18，代码替换全连接层 `fc`；对于 EfficientNet-B0，代码替换 `classifier` 中最后一层线性层：

```

1 def build_resnet18(num_classes: int = NUM_CLASSES, pretrained: bool = True)
  -> nn.Module:
2     weights = ResNet18_Weights.DEFAULT if pretrained else None
3     model = resnet18(weights=weights)
4     model.fc = nn.Linear(model.fc.in_features, num_classes)
5     return model
6
7
8 def build_efficientnet_b0(num_classes: int = NUM_CLASSES, pretrained: bool =
  True) -> nn.Module:
9     weights = EfficientNet_B0_Weights.DEFAULT if pretrained else None
10    model = efficientnet_b0(weights=weights)
11    in_features = model.classifier[-1].in_features
12    model.classifier[-1] = nn.Linear(in_features, num_classes)
13    return model

```

Listing 3: 分类模型输出层替换代码

分割任务使用 DeepLabV3-MobileNetV3。DeepLabV3 能够利用空洞卷积和多尺度上下文信息完成像素级预测，MobileNetV3 作为 backbone 可以减少一定计算开销。由于本实验只预测背景和宠物前景两类，因此需要将 DeepLabV3 的主分割头和辅助分割头输出通道数改为 2：

```

1 def _replace_last_conv(module: nn.Sequential, num_classes: int) -> None:

```

```

2     last_layer = module[-1]
3     if not isinstance(last_layer, nn.Conv2d):
4         raise TypeError(f"Expected final layer to be nn.Conv2d, got {type(
last_layer).__name__}")
5     module[-1] = nn.Conv2d(last_layer.in_channels, num_classes, kernel_size
=1)
6
7
8 def build_deeplabv3_mobilenet(num_classes: int = SEG_NUM_CLASSES, pretrained
: bool = True):
9     weights = DeepLabV3_MobileNet_V3_Large_Weights.DEFAULT if pretrained
else None
10    model = deeplabv3_mobilenet_v3_large(
11        weights=weights,
12        weights_backbone=None,
13        aux_loss=True,
14    )
15    _replace_last_conv(model.classifier, num_classes)
16    if model.aux_classifier is not None:
17        _replace_last_conv(model.aux_classifier, num_classes)
18    return model

```

Listing 4: 分割模型输出头替换代码

训练过程中，分类和分割任务均使用交叉熵损失，优化器使用 AdamW。分类模型在每个 epoch 后记录训练损失、验证损失、验证 Accuracy、验证 Macro-F1、学习率和耗时；分割模型在每个 epoch 后记录训练损失、验证损失、Pixel Accuracy、背景 IoU、前景 IoU、mIoU、Dice、学习率和耗时。

分类模型以验证集 Macro-F1 作为主要模型选择依据，当 Macro-F1 相同时再比较 Accuracy。分割模型以验证集 mIoU 作为主要模型选择依据，当 mIoU 相同时再比较 Dice。通过这种方式，可以在模型选择时同时考虑类别均衡表现和像素级分割质量。

4.1.4 评价与可视化方法

分类模型训练完成后，在官方测试集上计算 Accuracy、Macro-F1、Weighted-F1、Top-5 Accuracy 以及每类 precision、recall、F1。Accuracy 用于衡量整体分类正确率，Macro-F1 对每个类别等权平均，适合观察 37 类宠物品种上的整体均衡表现，Top-5 Accuracy 用于观察真实类别是否出现在模型预测概率最高的前 5 个类别中。

分割模型训练完成后，在官方测试集上计算 Pixel Accuracy、背景 IoU、前景 IoU、mIoU 和 Dice。Pixel Accuracy 反映所有像素的总体预测正确率，IoU 反映预测区域和真实区域的重叠程度，mIoU 对背景和前景 IoU 取平均，Dice 则重点反映宠物前景区域的重叠质量。

分割指标计算的核心逻辑如下：

```
1 prediction = prediction.reshape(-1)
2 target = target.reshape(-1)
3 pixel_accuracy = float((prediction == target).sum().item() / target.numel())
4
5 ious = []
6 for class_id in (0, 1):
7     pred_class = prediction == class_id
8     target_class = target == class_id
9     intersection = torch.logical_and(pred_class, target_class).sum().item()
10    union = torch.logical_or(pred_class, target_class).sum().item()
11    ious.append(1.0 if union == 0 else float(intersection / union))
12
13 foreground_intersection = torch.logical_and(prediction == 1, target == 1).
    sum().item()
14 foreground_total = (prediction == 1).sum().item() + (target == 1).sum().item()
15 dice = 1.0 if foreground_total == 0 else float((2 * foreground_intersection)
    / foreground_total)
```

Listing 5: 分割指标计算代码

可视化部分包括分类训练曲线、分割训练曲线、分类混淆矩阵、分类正确样例、分类错误样例、分割预测对比图和分割失败案例。所有图表和指标均由脚本根据真实运行输出生成，并保存到 `outputs/figures/` 和 `outputs/results/` 中，报告中的结果以这些文件为依据。

4.2 实验步骤

本实验按照数据检查、预处理验证、模型训练、测试评估、可视化整理和报告编译的顺序进行，具体步骤如下：

1. 准备项目运行环境，确认使用 `conda` 环境 `unet`，并在项目根目录下执行所有脚本。
2. 检查 Oxford-IIIT Pet 数据集是否可正常读取，统计分类类别数、样本数、训练集类别分布和 `trimap` 像素值。
3. 使用固定随机种子 42 将官方 `trainval split` 划分为训练集和验证集，官方 `test split` 保持为最终测试集，并保存划分索引。
4. 检查分类任务的图像预处理流程，确认训练集增强、验证集和测试集确定性预处理能够正常输出。

5. 检查分割任务的图像和 mask 预处理流程, 确认图像与 mask 空间变换同步, mask resize 未产生非法像素值。
6. 按二值前景分割规则检查 trimap 转换结果, 确认原始值 2 映射为背景, 原始值 1 和 3 映射为前景。
7. 训练 ResNet18 baseline 分类模型, 保存训练日志和验证集最优模型权重。
8. 训练 EfficientNet-B0 主分类模型, 保存训练日志和验证集最优模型权重。
9. 训练 DeepLabV3-MobileNetV3 分割模型, 保存训练日志和验证集最优模型权重。
10. 在官方测试集上评估 ResNet18 和 EfficientNet-B0, 生成分类指标 JSON、逐样本预测 CSV 和混淆矩阵图。
11. 在官方测试集上评估 DeepLabV3-MobileNetV3, 生成分割指标 JSON、测试摘要 CSV 和逐样本预测记录。
12. 绘制分类和分割训练曲线, 整理分类正确样例、分类错误样例、分割预测对比图和失败案例图。
13. 汇总 outputs/results/、outputs/logs/、outputs/figures/ 中的实验材料, 作为后续结果分析和报告插图的依据。

4.3 实验代码

本实验代码集中在 src/ 目录下, 训练日志、模型权重、结果文件和图片分别保存到 outputs/logs/、outputs/checkpoints/、outputs/results/ 和 outputs/figures/。主要代码模块及其作用如表 7 所示。

表 7: 主要代码模块说明

模块	主要文件	作用
配置模块	src/config.py	统一保存路径、随机种子、类别数、输入尺寸、batch size 和输出目录配置
数据处理模块	src/data_loaders.py、src/data_transforms.py、src/segmentation_utils.py	构建数据集、定义预处理和数据增强, 完成 trimap 到二值 mask 的转换
分类模型模块	src/models_cls.py	构建 ResNet18 baseline 和 EfficientNet-B0 主分类模型
分割模型模块	src/models_seg.py	构建 DeepLabV3-MobileNetV3 前景二分类分割模型
训练模块	src/train_cls.py、src/train_seg.py	执行分类模型和分割模型训练, 记录日志并保存最优权重
评估模块	src/eval_cls.py、src/eval_seg.py	在测试集上计算分类和分割指标, 保存预测结果
可视化模块	src/plot_cls_training_curve.py、src/plot_seg_training_curve.py、src/visualize_seg_predictions.py	绘制训练曲线、混淆矩阵和分割预测可视化图
结果汇总模块	src/summarize_cls_eval.py	汇总分类评估结果, 生成报告可引用的表格材料

4.3.1 数据处理模块重要代码

该模块主要用于完成数据集读取、固定划分、图像预处理和 mask 转换。分类数据集读取宠物品种标签, 分割数据集读取 trimap 标注, 二者使用相同的 train/val/test 划分口径。

数据读取和固定划分代码如下:

```

1 class OxfordPetClassificationDataset(Dataset):
2     def __init__(self, root=DATA_DIR, split="train", img_size=224,
3                 val_ratio=0.2, seed=SEED, download=False):
4         source_split = "test" if split == "test" else "trainval"
5         self.dataset = OxfordIIITPet(
6             root=str(root),
7             split=source_split,
8             target_types="category",
9             download=download,
10        )
11        self.indices = _select_indices(len(self.dataset), split, val_ratio,
12                                     seed)
13        self.transform = build_classification_transform(split, img_size=
14                                     img_size)

```

Listing 6: 分类数据集读取与固定划分代码

分割预处理需要同时处理图像和 `mask`。图像使用双线性插值，`mask` 使用最近邻插值，随后将 `trimap` 转为二值 `mask`：

```

1 image = F.resized_crop(
2     image, i, j, h, w,
3     size=[self.img_size, self.img_size],
4     interpolation=InterpolationMode.BILINEAR,
5     antialias=True,
6 )
7 mask = F.resized_crop(
8     mask, i, j, h, w,
9     size=[self.img_size, self.img_size],
10    interpolation=InterpolationMode.NEAREST,
11 )
12
13 mask_tensor = torch.as_tensor(np.array(mask, dtype=np.int64), dtype=torch.
14                               long)
15 mask_tensor = convert_trimap_to_binary_mask(mask_tensor)

```

Listing 7: 分割图像与 `mask` 同步预处理代码

这段代码保证了图像和 `mask` 的裁剪区域一致，同时避免 `mask` 在 `resize` 后出现非整数类别标签。

4.3.2 分类模型模块重要代码

该模块主要用于构建 37 类宠物品种分类模型。由于 `TorchVision` 的预训练分类模型默认输出 ImageNet 1000 类，因此需要将最后分类层替换为 37 类输出。

ResNet18 baseline 的关键代码如下：

```
1 weights = ResNet18_Weights.DEFAULT if pretrained else None
2 model = resnet18(weights=weights)
3 model.fc = nn.Linear(model.fc.in_features, num_classes)
```

Listing 8: ResNet18 输出层替换代码

EfficientNet-B0 主分类模型的关键代码如下：

```
1 weights = EfficientNet_B0_Weights.DEFAULT if pretrained else None
2 model = efficientnet_b0(weights=weights)
3 in_features = model.classifier[-1].in_features
4 model.classifier[-1] = nn.Linear(in_features, num_classes)
```

Listing 9: EfficientNet-B0 输出层替换代码

两段代码都保留了预训练模型的通用视觉特征提取部分，只替换与本任务类别数相关的输出层，使模型能够适配 Oxford-IIIT Pet 的 37 类分类任务。

4.3.3 分割模型模块重要代码

该模块主要用于构建宠物前景二分类语义分割模型。TorchVision 的 DeepLabV3-MobileNetV3 返回字典形式的输出，其中主输出位于 `output["out"]`，形状为 `[B, num_classes, H, W]`。本实验将 `num_classes` 设为 2，分别对应背景和宠物前景。

模型输出头替换代码如下：

```
1 def _replace_last_conv(module: nn.Sequential, num_classes: int) -> None:
2     last_layer = module[-1]
3     module[-1] = nn.Conv2d(last_layer.in_channels, num_classes, kernel_size
4                             =1)
5
6 model = deeplabv3_mobilenet_v3_large(
7     weights=weights,
8     weights_backbone=None,
9     aux_loss=True,
10 )
11 _replace_last_conv(model.classifier, num_classes)
12 if model.aux_classifier is not None:
13     _replace_last_conv(model.aux_classifier, num_classes)
```

Listing 10: DeepLabV3-MobileNetV3 输出头替换代码

这段代码将主分割头和辅助分割头都改为 2 类输出，使模型可以对每个像素预测背景或宠物前景。

4.3.4 训练与评估模块重要代码

该模块主要用于执行模型训练、记录训练过程、保存验证集最优模型，并在测试集上计算最终指标。分类模型和分割模型都按 `epoch` 训练，并在每个 `epoch` 后进行验证。

分类训练中，模型保存依据为验证集 Macro-F1；当 Macro-F1 相同时，再比较 Accuracy：

```
1 improved = (val_macro_f1 > best_macro_f1) or (  
2     val_macro_f1 == best_macro_f1 and val_accuracy > best_accuracy  
3 )  
4 if improved:  
5     best_macro_f1 = val_macro_f1  
6     best_accuracy = val_accuracy  
7     best_epoch = epoch  
8     torch.save(  
9         {  
10             "epoch": epoch,  
11             "model_state_dict": model.state_dict(),  
12             "optimizer_state_dict": optimizer.state_dict(),  
13             "val_accuracy": val_accuracy,  
14             "val_macro_f1": val_macro_f1,  
15         },  
16         checkpoint_path,  
17     )
```

Listing 11: 分类最佳模型保存代码

分割训练中，模型保存依据为验证集 mIoU；当 mIoU 相同时，再比较 Dice：

```
1 val_miou = float(val_metrics["miou"])  
2 val_dice = float(val_metrics["dice"])  
3 improved = (val_miou > best_miou) or (val_miou == best_miou and val_dice >  
4     best_dice)  
4 if improved:  
5     best_miou = val_miou  
6     best_dice = val_dice  
7     best_epoch = epoch  
8     torch.save(  
9         {  
10             "epoch": epoch,  
11             "model_state_dict": model.state_dict(),  
12             "optimizer_state_dict": optimizer.state_dict(),  
13             "val_miou": val_metrics["miou"],  
14             "val_dice": val_metrics["dice"],  
15         },
```

```

16     checkpoint_path,
17 )

```

Listing 12: 分割最佳模型保存代码

分类评估脚本 `src/eval_cls.py` 加载最优权重后，在测试集上执行前向推理，并保存指标、逐样本预测结果和混淆矩阵。分割评估脚本 `src/eval_seg.py` 加载最优权重后，计算像素级指标，并保存逐样本分割预测摘要。报告中的 Accuracy、Macro-F1、mIoU、Dice 等数值均以这些脚本生成的真实结果文件为准。

五、实验过程原始记录

本节对实验过程中形成的主要原始记录进行整理，包括数据划分记录、预处理检查结果、训练日志、测试指标和可视化图表等内容。相关文件统一保存在项目的 `outputs/` 目录下，其中 `outputs/logs/` 用于保存训练过程日志，`outputs/results/` 用于保存数据统计、指标计算和逐样本预测结果，`outputs/figures/` 用于保存数据样例、训练曲线、混淆矩阵和分割结果图，`outputs/checkpoints/` 用于保存训练过程中得到的最佳模型权重。通过这些记录，可以对实验过程和最终结果进行复查。

5.1 数据样例与类别分布记录

本实验使用 Oxford-IIIT Pet 数据集作为实验对象。该数据集包含 37 类宠物品种图像，同时提供品种分类标签和 `trimap` 分割标注，能够同时支持图像分类和前景分割两个任务。实验中使用固定随机种子 42 对官方 `trainval` 部分进行训练集和验证集划分，官方 `test` 部分保持独立，只在最终测试阶段使用。

具体数据划分情况见表 8。划分索引已保存到 `outputs/results/split_indices_seed42.js`。后续分类训练、分割训练和模型评估均复用这一划分方式，从而保证不同实验之间的数据来源一致。

表 8: 数据划分记录

划分	样本数	用途
官方 <code>trainval</code>	3680	原始训练验证数据
<code>train</code>	2944	模型训练
<code>val</code>	736	训练过程验证和最佳模型选择
<code>test</code>	3669	最终测试评估

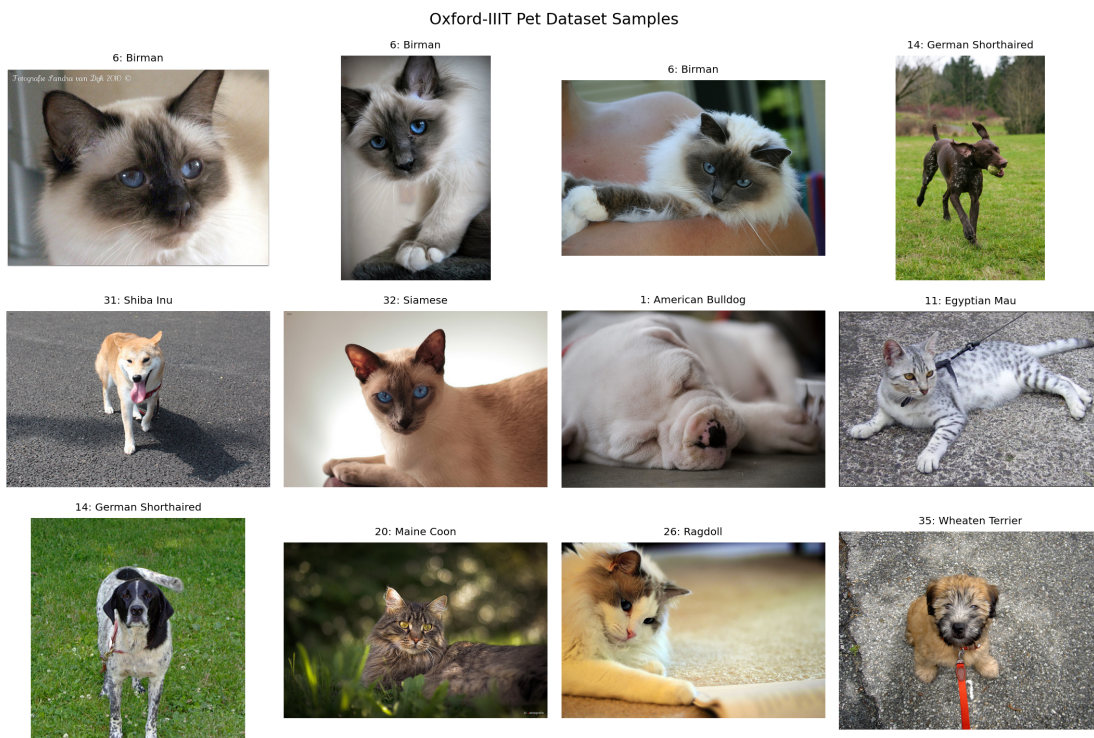


图 2: Oxford-IIIT Pet 数据集样例

图 2 展示了数据集中不同宠物品种的图像样例。从样例可以看到，宠物图像在姿态、背景、拍摄距离和光照条件上差异较大；同时，部分猫狗品种在毛色、脸部结构和体型上较为相似，因此本实验中的分类任务具有一定的细粒度识别难度。

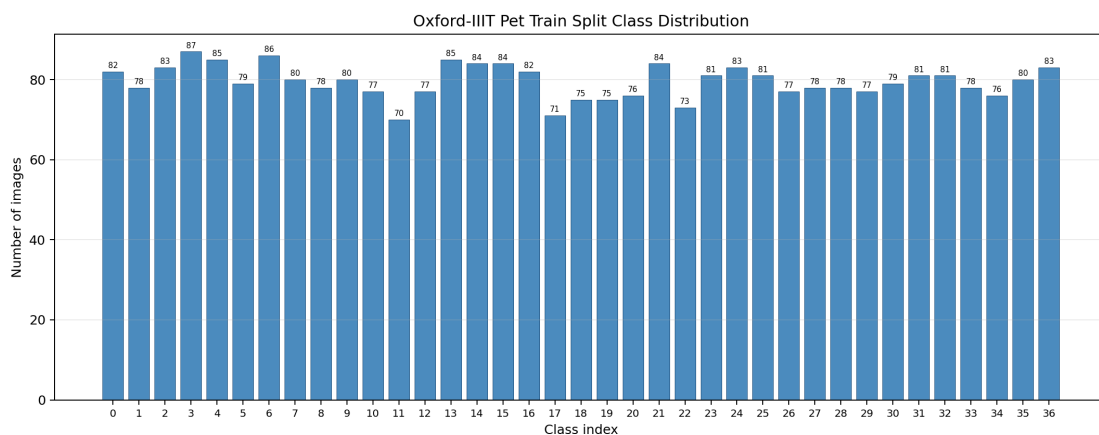


图 3: 训练集类别分布

图 3 展示了训练集中的类别分布情况。Oxford-IIIT Pet 数据集整体上较为均衡，但在固定随机划分后，不同类别在训练集和验证集中的样本数量仍会存在小幅波动。因此，在分类任务中，除整体 Accuracy 外，本实验还记录 Macro-F1，以便观察模型在各类别上的平均表现。

本部分原始记录文件见表 9。

表 9: 数据相关原始记录文件

记录内容	文件路径
数据集统计摘要	outputs/results/data_summary.json
类别分布 CSV	outputs/results/class_distribution.csv
固定划分索引	outputs/results/split_indices_seed42.json
数据样例图	outputs/figures/fig_dataset_samples.png
类别分布图	outputs/figures/fig_class_distribution.png

5.2 Mask 转换与预处理检查记录

Oxford-IIIT Pet 数据集提供的 trimap 标注包含 3 种像素值：1 表示宠物前景，2 表示背景，3 表示边界区域。由于本实验的分割目标是区分宠物前景和背景，因此需要先将原始 trimap 转换为二值 mask。具体做法是将原始像素值 2 映射为背景 0，将原始像素值 1 和 3 映射为前景 1。

该转换规则见表 10。转换完成后，分割模型只需要预测背景和前景两类，后续计算背景 IoU、前景 IoU、mIoU 和 Dice 时也均以该二值 mask 作为标签依据。

表 10: trimap 到二值 mask 的转换规则

原始 trimap 值	原始含义	二值 mask 值	二值含义
1	宠物前景	1	前景
2	背景	0	背景
3	边界区域	1	前景

Step 4 Mask Conversion Samples



图 4: 原始 trimap 与二值前景 mask 转换样例

图 4 展示了原始图像、原始 trimap 和二值 mask 的对比样例。通过该图可以检查 mask 转换是否正确，也可以直观观察边界像素并入前景后形成的标注区域。

分类任务和分割任务使用不同的输入尺寸。分类模型输入尺寸为 224×224 ，分割模型输入尺寸为 320×320 。在分割任务中，图像 resize 使用普通图像插值方式，mask resize 使用最近邻插值，以避免离散类别标签在缩放过程中产生非法像素值。训练阶段的随机裁剪和水平翻转也对图像和 mask 同步执行，从而保证像素级标签与图像内

容保持对齐。

本部分原始记录文件见表 11。

表 11: mask 转换与预处理检查文件

记录内容	文件路径
trimap 转换规则	outputs/results/trimap_rule_step4.json
mask 转换抽查记录	outputs/results/mask_conversion_check.json
数据预处理检查记录	outputs/results/preprocessing_check.json
DataLoader 检查记录	outputs/results/dataloader_check.json
mask 样例图	outputs/figures/fig_mask_samples.png
预处理检查图	outputs/figures/fig_preprocessing_check.png

5.3 训练过程记录

本实验共完成三个核心训练实验，分别为 ResNet18 分类 baseline、EfficientNet-B0 分类主模型和 DeepLabV3-MobileNetV3 前景分割模型。三个实验均使用固定数据划分和固定随机种子，训练过程中的主要指标以 CSV 文件形式保存，便于后续绘制曲线和复核结果。

训练过程概况见表 12。分类模型以验证集 Macro-F1 作为主要模型选择依据，分割模型以验证集 mIoU 作为主要模型选择依据，并在验证指标最优时保存对应权重。

表 12: 训练过程记录概况

实验编号	任务	模型	输入尺寸	训练 epoch	最佳 epoch	训练日志
Exp-1	分类	ResNet18	224	20	15	outputs/logs/resnet18_cls_log.csv
Exp-2	分类	EfficientNet-B0	224	25	13	outputs/logs/efficientnet_b0_cls_log.csv
Exp-3	分割	DeepLabV3-MobileNetV3	320	30	23	outputs/logs/deeplabv3_mobilenet_seg_log.csv

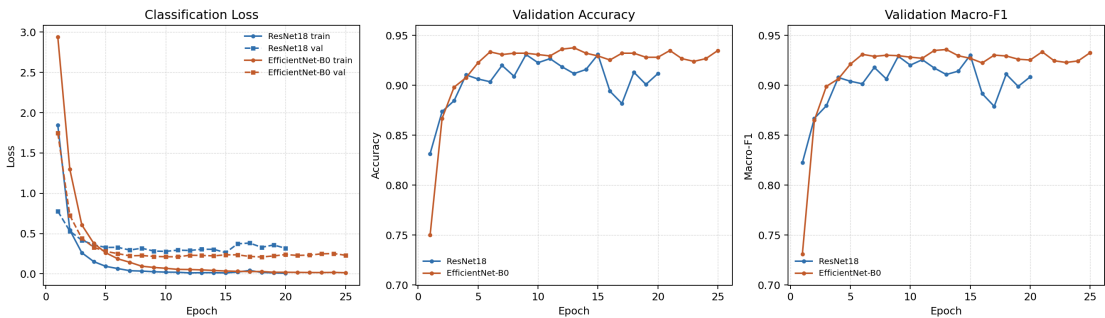


图 5: ResNet18 与 EfficientNet-B0 分类训练曲线

图 5 展示了 ResNet18 和 EfficientNet-B0 的分类训练曲线。训练日志中记录了每个 epoch 的训练损失、验证损失、验证 Accuracy、验证 Macro-F1、学习率和耗时。从曲线变化可以看出，两个分类模型在训练过程中整体能够逐步提升验证指标，没有出现明显的 loss 发散现象，说明训练过程基本稳定。

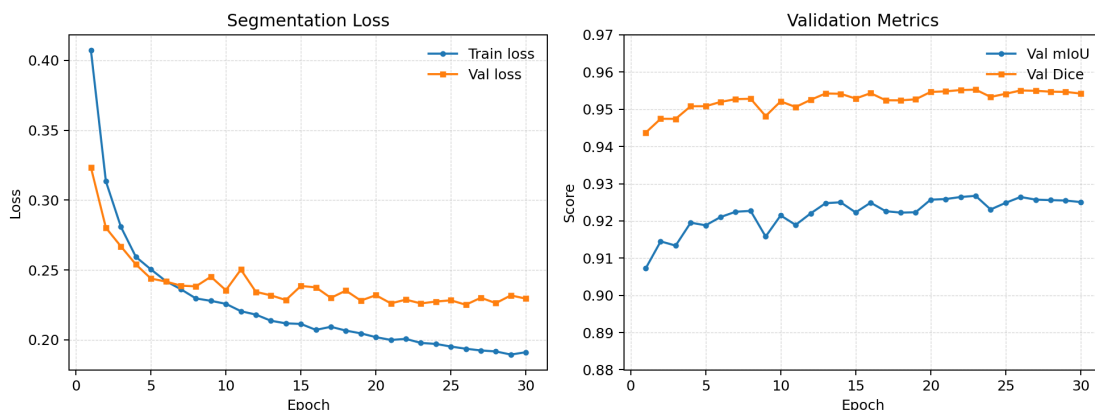


图 6: DeepLabV3-MobileNetV3 分割训练曲线

图 6 展示了 DeepLabV3-MobileNetV3 的分割训练曲线。分割训练日志记录了训练损失、验证损失、Pixel Accuracy、背景 IoU、前景 IoU、mIoU 和 Dice 等指标。根据训练记录,最佳验证 mIoU 出现在第 23 个 epoch,对应权重保存为 `outputs/checkpoints/best_seg_deep` 本部分原始记录文件见表 13。

表 13: 训练日志与最佳权重文件

任务	记录类型	文件路径
ResNet18 分类	训练日志	<code>outputs/logs/resnet18_cls_log.csv</code>
ResNet18 分类	最佳权重	<code>outputs/checkpoints/best_cls_resnet18.pth</code>
EfficientNet-B0 分类	训练日志	<code>outputs/logs/efficientnet_b0_cls_log.csv</code>
EfficientNet-B0 分类	最佳权重	<code>outputs/checkpoints/best_cls_efficientnet_b0.pth</code>
DeepLabV3-MobileNetV3 分割	训练日志	<code>outputs/logs/deeplabv3_mobilenet_seg_log.csv</code>
DeepLabV3-MobileNetV3 分割	最佳权重	<code>outputs/checkpoints/best_seg_deeplabv3_mobilenet.pth</code>

5.4 测试数据与计算结果记录

模型训练完成后,本实验在官方测试集上分别评估两个分类模型和一个分割模型。测试集共包含 3669 张图像,只用于最终评估,不参与训练过程和最佳模型选择。

分类任务的测试结果见表 14。ResNet18 在测试集上的 Accuracy 为 0.883074, Macro-F1 为 0.881759; EfficientNet-B0 在测试集上的 Accuracy 为 0.899700, Macro-F1 为 0.898173。两个模型的 Top-5 Accuracy 均高于 0.98,说明在多数情况下,真实类别能够出现在模型预测概率最高的前几个候选类别中。

表 14: 分类任务测试结果记录

模型	Accuracy	Macro-F1	Weighted-F1	Top-5 Accuracy	Test Loss
ResNet18	0.883074	0.881759	0.882539	0.986645	0.418459
EfficientNet-B0	0.899700	0.898173	0.898878	0.995639	0.344536

上述分类测试指标来自 `outputs/results/cls_ablation_test_summary.csv`。逐样本预测结果分别保存为 `outputs/results/cls_predictions_resnet18_test.csv`

和 `outputs/results/cls_predictions_efficientnet_b0_test.csv`, 可用于进一步查看每张测试图像的真实类别、预测类别和预测置信度。

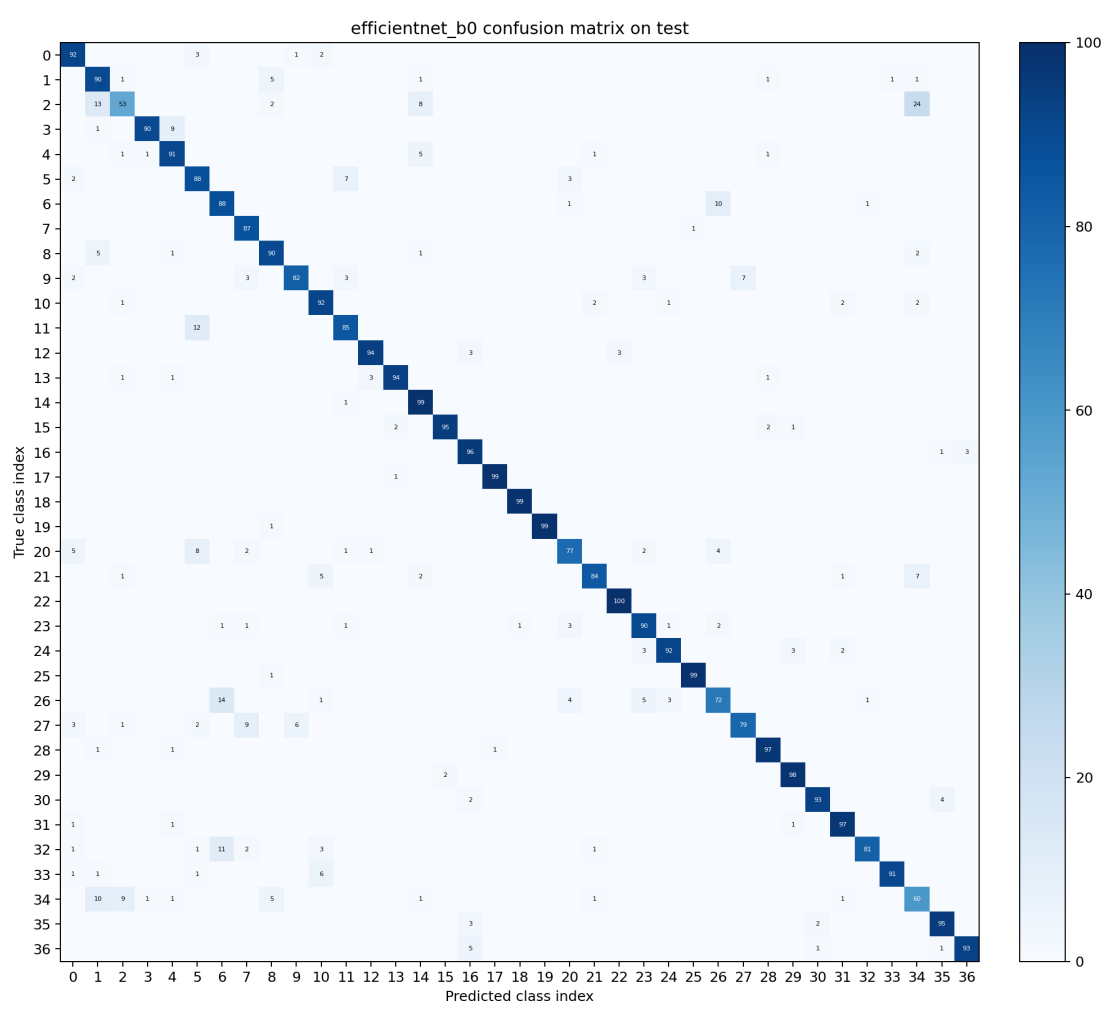


图 7: EfficientNet-B0 测试集混淆矩阵记录

图 7 为 EfficientNet-B0 在测试集上的混淆矩阵。该图记录了 37 类宠物品种之间的预测分布情况，可用于观察哪些类别更容易被模型混淆，并为后续结果分析提供依据。

分割任务的测试结果见表 15。DeepLabV3-MobileNetV3 在测试集上的 Pixel Accuracy 为 0.961981，前景 IoU 为 0.913212，mIoU 为 0.924921，Dice 为 0.954638。这些结果来自 `outputs/results/seg_metrics_deeplabv3_mobilenet.json` 和 `outputs/results/seg`

表 15: 分割任务测试结果记录

模型	Pixel Accuracy	Background IoU	Foreground IoU	mIoU	Dice	Test Loss
DeepLabV3-MobileNetV3	0.961981	0.936630	0.913212	0.924921	0.954638	0.116487

在分割评估过程中，程序还记录了有效像素数、背景/前景交并集、预测前景像素数和真实前景像素数等中间计算量。这些数值保存在 `outputs/results/seg_metrics_deeplabv3_`

中，可用于复查 mIoU 和 Dice 等指标的计算过程。

5.5 模型输出样例记录

除数值指标外，本实验还生成了分类和分割任务的可视化输出，用于更直观地观察模型预测效果。

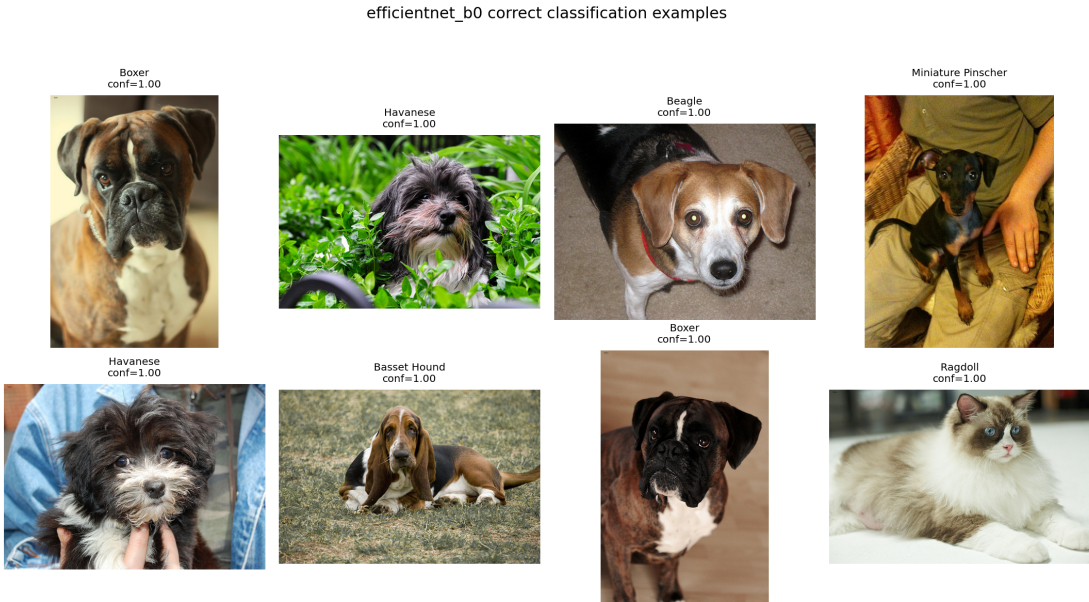


图 8: EfficientNet-B0 分类正确样例

图 8 展示了 EfficientNet-B0 在测试集上的部分分类正确样例。样例中包含不同品种、不同背景和不同姿态的宠物图像，可以反映模型在常规样本上的识别情况。

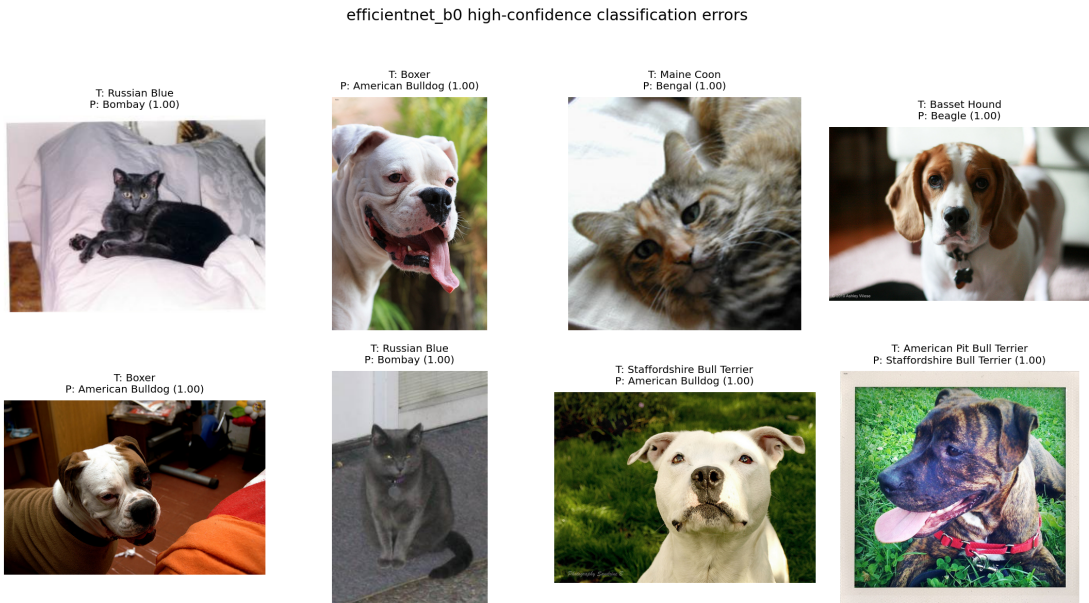


图 9: EfficientNet-B0 高置信度分类错误样例记录

图 9 展示了 EfficientNet-B0 的部分高置信度分类错误样例。这些样例主要作为后续错误分析的原始材料，具体错误原因可在第六节中结合品种外观相似性、拍摄角度和背景干扰等因素进一步讨论。



图 10: DeepLabV3-MobileNetV3 分割预测对比记录

图 10 展示了 DeepLabV3-MobileNetV3 在测试集上的分割预测对比，包括原图、真实 mask 和预测 mask。通过该图可以观察模型对宠物主体区域的定位效果，也可以看到模型在边界、遮挡、浅色背景等情况下可能出现的预测差异。

本部分原始记录文件见表 16。

表 16: 模型输出样例相关记录文件

记录内容	文件路径
EfficientNet-B0 分类样例记录	outputs/results/cls_examples_efficientnet_b0.json
EfficientNet-B0 测试集逐样本预测	outputs/results/cls_predictions_efficientnet_b0_test.csv
ResNet18 测试集逐样本预测	outputs/results/cls_predictions_resnet18_test.csv
分割测试集逐样本预测	outputs/results/seg_predictions_deeplabv3_mobilenet_test.csv
分割可视化样例记录	outputs/results/seg_visualized_samples.json
分割困难样例分析记录	outputs/results/seg_failure_analysis_step10.md

5.6 实验记录索引

为便于复核实验过程，表 17 汇总了本实验中较为重要的原始记录文件。这些文件覆盖了数据划分、模型训练、测试指标、逐样本预测和可视化结果等内容，可以作为后续报告分析和实验复现的依据。

表 17: 主要实验记录文件索引

类别	记录内容	文件路径
数据	数据集统计摘要	outputs/results/data_summary.json
数据	固定划分索引	outputs/results/split_indices_seed42.json
数据	类别分布统计	outputs/results/class_distribution.csv
分类训练	ResNet18 训练日志	outputs/logs/resnet18_cls_log.csv
分类训练	EfficientNet-B0 训练日志	outputs/logs/efficientnet_b0_cls_log.csv
分割训练	DeepLabV3-MobileNetV3 训练日志	outputs/logs/deeplabv3_mobilenet_seg_log.csv
分类评估	分类测试汇总	outputs/results/cls_ablation_test_summary.csv
分类评估	ResNet18 分类指标	outputs/results/cls_metrics_resnet18.json
分类评估	EfficientNet-B0 分类指标	outputs/results/cls_metrics_efficientnet_b0.json
分割评估	分割测试指标	outputs/results/seg_metrics_deeplabv3_mobilenet.json
分割评估	分割测试汇总	outputs/results/seg_summary_deeplabv3_mobilenet_test.csv
可视化	分类训练曲线	outputs/figures/fig_cls_training_curve.png
可视化	分割训练曲线	outputs/figures/fig_seg_training_curve.png
可视化	EfficientNet-B0 混淆矩阵	outputs/figures/fig_confusion_matrix.png
可视化	分割预测对比图	outputs/figures/fig_seg_predictions.png

综上，本实验的训练日志、测试指标、逐样本预测结果和可视化图表均已保存为可追溯文件。本节主要完成实验过程原始记录的整理，第六节将在这些记录的基础上进一步分析分类模型对比结果、类别混淆现象、分割模型表现以及实验结论。

六、实验结果、分析和结论

本实验以 Oxford-IIIT Pet 数据集为实验对象，完成了宠物品种分类和宠物前景语义分割两个任务。分类任务需要对 37 个宠物品种进行识别，实验中选取 ResNet18 作为 baseline，并使用 EfficientNet-B0 作为主要分类模型；分割任务则将原始 trimap 标注转换为宠物前景与背景二分类 mask，并使用 DeepLabV3-MobileNetV3 完成像素级预测。为了保证结果具有可比性，实验过程中使用固定随机种子 42 进行数据划分，官方测试集只用于最终评估，不参与训练和模型选择。

6.1 分类结果

分类任务在官方测试集上使用 Accuracy、Macro-F1、Weighted-F1 和 Top-5 Accuracy 作为评价指标。ResNet18 与 EfficientNet-B0 的测试结果如表 18 所示。

表 18: 分类模型测试集结果

模型	Accuracy	Macro-F1	Weighted-F1	Top-5 Accuracy
ResNet18	0.883074	0.881759	0.882539	0.986645
EfficientNet-B0	0.899700	0.898173	0.898878	0.995639

由表 18 可以看出，EfficientNet-B0 在测试集上的 Accuracy 为 0.899700，Macro-F1 为 0.898173，均高于 ResNet18 的 Accuracy 0.883074 和 Macro-F1 0.881759。相比 ResNet18，EfficientNet-B0 的 Accuracy 提升了 0.016626，Macro-F1 提升了 0.016414。虽然从数值上看提升幅度不是特别大，但 Oxford-IIIT Pet 数据集包含 37 个宠物品种，许多品种之间差异较细，例如毛色、脸型、耳朵形状和身体比例都可能非常接近，因此该提升仍能说明 EfficientNet-B0 在细粒度分类任务中具有更好的特征表达能力。

从 Top-5 Accuracy 看，两个模型的结果都比较高，其中 ResNet18 为 0.986645，EfficientNet-B0 为 0.995639。这说明在多数样本中，即使模型的 Top-1 预测结果不完全正确，真实类别也往往仍然位于前几个候选类别中。也就是说，模型通常能够判断出样本的大致类别范围，但在外观接近的品种之间仍容易出现排序错误。

Accuracy 和 Macro-F1 的数值比较接近，说明模型没有明显依赖某几个样本数较多或较容易识别的类别，而是在整体类别上保持了相对均衡的表现。由于 Macro-F1 会对每个类别赋予相同权重，因此它比单纯的 Accuracy 更能反映模型在 37 个类别上的平均识别能力。EfficientNet-B0 在这两个指标上均优于 ResNet18，说明其整体预测能力和类别均衡表现都更好。

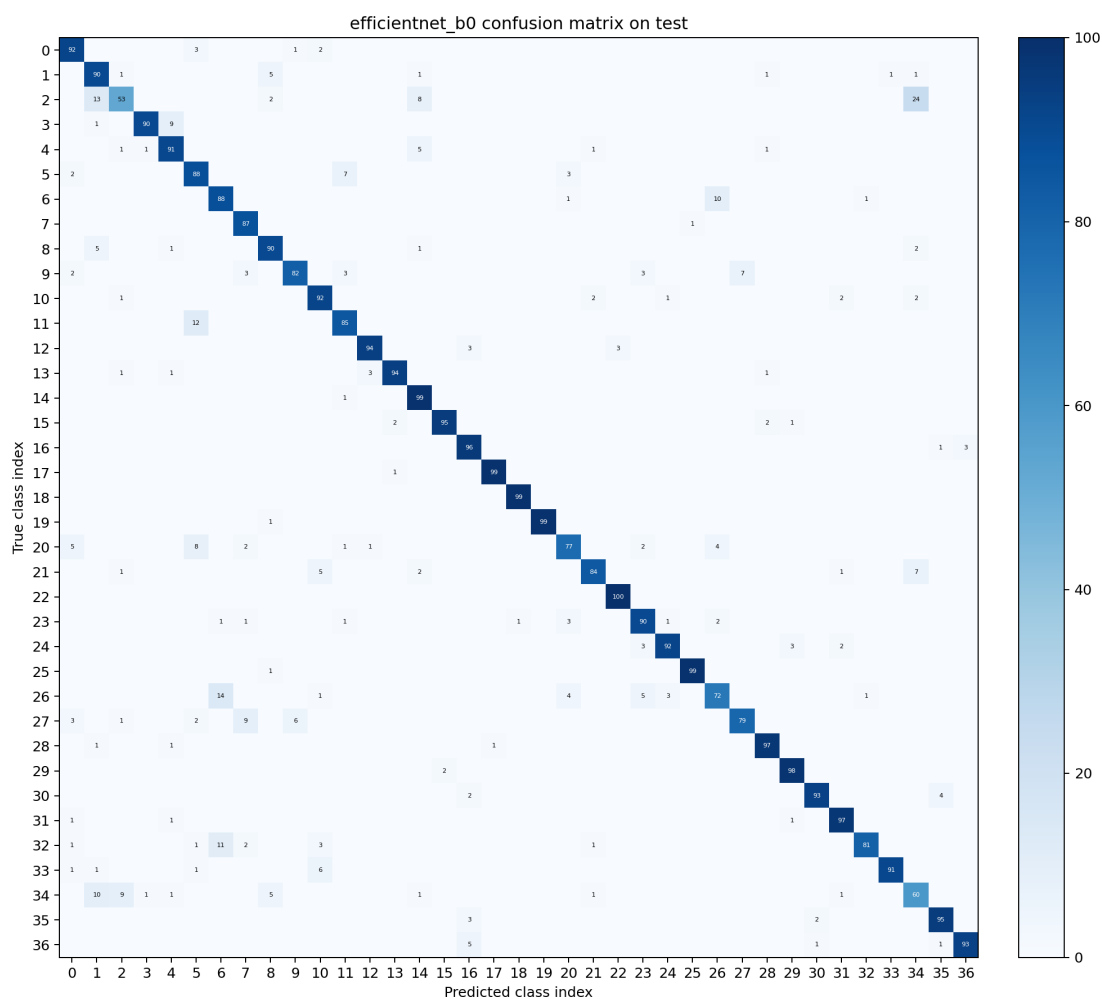


图 11: EfficientNet-B0 在测试集上的 37 类混淆矩阵

结合图 11 和分类失败样例可以进一步发现，分类错误主要集中在外观相似的宠物品种之间。例如 American Pit Bull Terrier、Staffordshire Bull Terrier 和 American Bulldog 等犬类在头部轮廓、体型和毛色上较为接近，模型容易混淆；Ragdoll 与 Birman 等长毛猫品种在毛色分布和面部特征上相似，也会出现互相误判；Russian Blue 与 Bombay 等深色短毛猫在光照不足或纹理不明显的图像中也较难区分。这说明宠物品种分类不仅是普通图像分类问题，还具有一定细粒度识别特点，模型需要捕捉更局部、更细致的视觉差异。

表 19: EfficientNet-B0 主要混淆类别

真实类别	预测类别	错误数量
American Pit Bull Terrier	Staffordshire Bull Terrier	24
Ragdoll	Birman	14
American Pit Bull Terrier	American Bulldog	13
Egyptian Mau	Bengal	12
Siamese	Birman	11
Staffordshire Bull Terrier	American Bulldog	10
Birman	Ragdoll	10

6.2 分割结果

分割任务使用 DeepLabV3-MobileNetV3 对宠物前景进行二分类语义分割。测试阶段计算 Pixel Accuracy、Background IoU、Foreground IoU、mIoU 和 Dice 等指标，结果如表 20 所示。

表 20: 分割模型测试集结果

模型	Pixel Acc.	Background IoU	Foreground IoU	mIoU	Dice
DeepLabV3-MobileNetV3	0.961981	0.936630	0.913212	0.924921	0.954638

从表 20 可以看出，DeepLabV3-MobileNetV3 在官方测试集上取得了 Pixel Accuracy 0.961981、mIoU 0.924921 和 Dice 0.954638。Pixel Accuracy 较高，说明模型在整体像素分类上具有较好的准确率；mIoU 和 Dice 也达到较高水平，说明预测出的宠物前景区域与真实 mask 之间具有较好的重合程度。

进一步观察类别 IoU 可以发现，背景 IoU 为 0.936630，前景 IoU 为 0.913212，前景 IoU 略低于背景 IoU。这一现象符合任务特点：背景区域通常面积较大，边界相对简单，而宠物前景包含毛发、耳朵、尾巴、四肢等细小结构，边缘形状更加复杂，也更容易受到遮挡、裁剪和背景颜色的影响。因此，模型在前景边界处更容易出现漏分或误分。



图 12: DeepLabV3-MobileNetV3 测试集分割预测对比

从图 12 的分割预测可视化结果来看，大多数样本中模型能够比较完整地覆盖宠物主体，并能将宠物与背景区分开来。分割效果较好的样本通常具有主体清晰、轮廓完整、前景和背景对比明显等特点。相反，低 mIoU 样本通常出现在浅色宠物与浅色背景接近、长毛边界复杂、图像近景裁剪严重或标注边界本身较模糊的情况下。这些

现象说明，模型已经能够学习到宠物主体的整体位置，但在精细边界和低对比区域仍然存在不足。

表 21: 分割低 mIoU 困难样例

测试集索引	mIoU	Dice	主要现象
2005	0.0189	0.0728	近景猫脸与真实 mask 区域差异明显，主体范围和边界定位失败。
2302	0.0473	0.1729	白色宠物与浅色背景接近，裁剪区域导致前景范围偏差。
2308	0.0815	0.2802	长毛白色宠物处于草地背景中，毛发边界复杂，预测区域与真实 mask 不一致。

总体来看，DeepLabV3-MobileNetV3 在本实验的宠物前景分割任务中取得了较好的效果。该模型结构相对轻量，同时能保持较高的 mIoU 和 Dice，说明使用预训练语义分割模型进行迁移学习可以有效完成本数据集上的前景分割任务。

6.3 轻量消融实验

为了比较不同分类模型的效果，本实验对 ResNet18 baseline 和 EfficientNet-B0 主模型进行了轻量消融对比。两种模型使用相同的数据划分、相同的官方测试集和相同的输入尺寸，并且都加载 ImageNet 预训练权重进行微调。实验结果如表 22 所示。

表 22: ResNet18 与 EfficientNet-B0 对比

模型	最佳 epoch	验证 Macro-F1	测试 Accuracy	测试 Macro-F1	训练总耗时
ResNet18	15	0.930059	0.883074	0.881759	765.40 s
EfficientNet-B0	13	0.935778	0.899700	0.898173	1010.81 s

从表 22 可以看出，EfficientNet-B0 的验证集 Macro-F1 和测试集 Macro-F1 均高于 ResNet18，说明 EfficientNet-B0 在本分类任务中表现更好。EfficientNet-B0 在网络深度、宽度和分辨率之间进行了较均衡的设计，因此能够提取到更有效的图像特征，在宠物细粒度分类任务中具有一定优势。

同时也可以看到，EfficientNet-B0 的训练总耗时约为 1010.81 秒，高于 ResNet18 的 765.40 秒。这说明模型性能提升并不是没有代价的，EfficientNet-B0 需要更多训练时间。综合来看，ResNet18 结构较简单、训练速度较快，适合作为 baseline；EfficientNet-B0 指标更高，更适合作为本实验的主分类模型。

6.4 失败案例分析

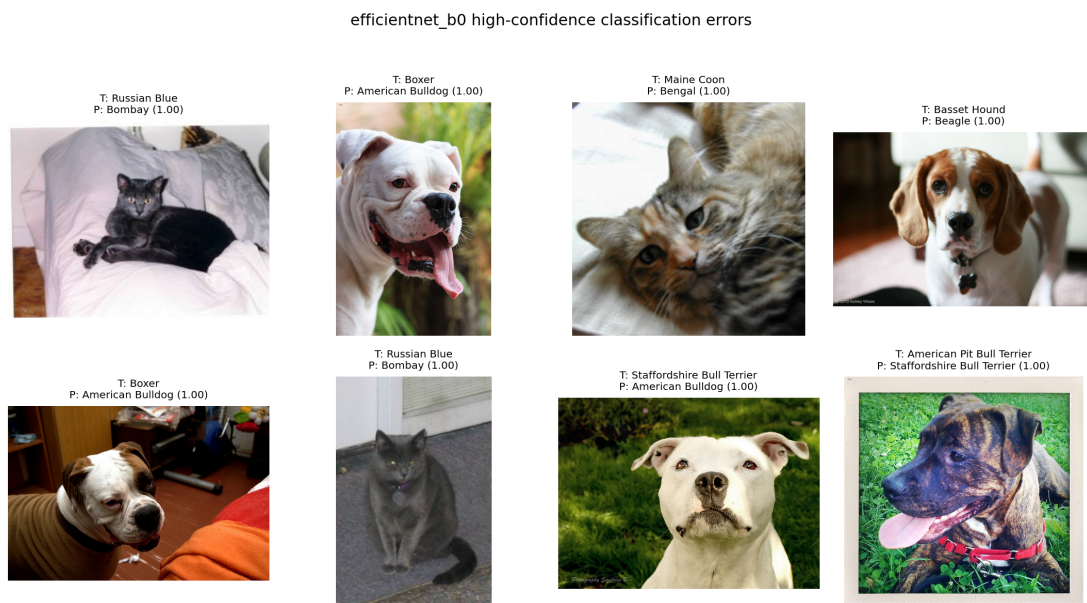


图 13: EfficientNet-B0 高置信度分类错误样例

从图 13 的分类失败案例看，模型的主要错误来自细粒度类别之间的相似性。犬类中，American Pit Bull Terrier、Staffordshire Bull Terrier 和 American Bulldog 等类别之间存在较多混淆，这些品种在头部形状、短毛纹理和身体比例上差异较小。猫类中，Ragdoll 与 Birman、Russian Blue 与 Bombay 等类别也容易被误判，主要原因是毛色、面部纹理和拍摄角度会影响模型对类别特征的判断。此外，部分图像存在遮挡、裁剪、姿态特殊或背景复杂等情况，也会增加分类难度。

从分割失败案例看，模型的错误主要集中在边界复杂和前景背景对比不足的样本上。长毛宠物的毛发边缘不规则，模型预测的 mask 边界很难完全贴合真实标注；浅色宠物位于浅色背景中时，前景与背景差异不明显，容易出现漏分；近景裁剪样本中，宠物主体范围不完整，模型难以判断前景的完整边界。另外，Oxford-IIIT Pet 的 trimap 标注在边缘区域本身具有一定模糊性，模型预测边界与标注边界不完全一致也会影响 mIoU。

这些失败案例说明，当前模型已经能够完成宠物主体识别和前景提取，但对于更细致的局部判别和复杂边界处理仍有提升空间。分类任务需要进一步提高模型对局部结构、毛发纹理和面部细节的关注；分割任务则需要进一步改善边界区域和低对比场景下的预测效果。

6.5 结论与改进方向

本实验完成了 Oxford-IIIT Pet 数据集上的 37 类宠物品种分类和宠物前景二分类语义分割。分类任务中，ResNet18 baseline 在测试集上取得 Accuracy 0.883074、Macro-

F1 0.881759; EfficientNet-B0 主模型取得 Accuracy 0.899700、Macro-F1 0.898173, 整体表现优于 ResNet18。分割任务中, DeepLabV3-MobileNetV3 在测试集上取得 Pixel Accuracy 0.961981、mIoU 0.924921 和 Dice 0.954638, 能够较稳定地提取宠物前景区域。

通过本实验可以看出, 迁移学习适用于 Oxford-IIIT Pet 这类中等规模图像数据集。分类模型加载 ImageNet 预训练权重后, 可以在较少训练轮数内取得较好的分类效果; 分割模型使用预训练 DeepLabV3-MobileNetV3, 也能够二分前景分割任务上取得较高指标。同时, 实验结果也表明, 宠物品种分类中的相似类别区分和语义分割中的复杂边界处理仍然是主要难点。

后续可以从以下几个方面继续改进。首先, 在分类任务中可以引入更有针对性的数据增强、类别重采样或难例挖掘方法, 以改善易混淆类别的识别效果。其次, 可以尝试更强的分类 backbone、注意力机制或局部区域特征方法, 使模型更加关注脸部、毛发纹理和身体结构等关键细节。再次, 在分割任务中可以尝试更高输入分辨率、边界损失或后处理方法, 提高宠物毛发和轮廓区域的分割质量。最后, 还可以探索使用分割结果辅助分类, 例如先根据分割 mask 提取宠物主体区域, 再进行品种分类, 从而减少复杂背景对分类结果的干扰。

总体而言, 本实验较完整地完成了从数据处理、模型训练、指标评估到可视化分析的流程。通过对 ResNet18、EfficientNet-B0 和 DeepLabV3-MobileNetV3 的实验结果进行比较, 可以更直观地理解不同模型结构在分类和分割任务中的作用, 也加深了对迁移学习、评价指标、混淆矩阵和语义分割结果分析方法的认识。